

JANUS: A Dual-Constraint Generative Framework for Stealthy Node Injection Attacks

Jiahao Zhang
School of Software Engineering
Huazhong University of Science and
Technology
Wuhan, China
jiahao_zhang@hust.edu.cn

Xiaobing Pei*
School of Software Engineering
Huazhong University of Science and
Technology
Wuhan, China
xiaobingp@hust.edu.cn

Zhaokun Zhong
School of Software Engineering
Huazhong University of Science and
Technology
Wuhan, China
hust_zzk@hust.edu.cn

Wenqiang Hao
School of Software Engineering
Huazhong University of Science and
Technology
Wuhan, China
fivecoins@hust.edu.cn

Zhenghao Tang
School of Software Engineering
Huazhong University of Science and
Technology
Wuhan, China
zh_tang42@hust.edu.cn

Abstract

Graph Neural Networks (GNNs) have demonstrated remarkable performance across web-centric applications, yet they are vulnerable to sophisticated adversarial attacks, particularly node injection attacks. The success of such attacks heavily relies on their stealthiness, the ability to blend in with the original graph and evade detection. However, existing methods often achieve stealthiness by relying on indirect proxy metrics, lacking consideration for the fundamental characteristics of the injected content, or focusing only on imitating local structures, which leads to the problem of local myopia. To overcome these limitations, we propose a dual-constraint stealthy node injection framework, called Joint Alignment of Nodal and Universal Structures (JANUS). At the local level, we introduce a local feature manifold alignment strategy to achieve geometric consistency in the feature space. At the global level, we incorporate structured latent variables and maximize the mutual information with the generated structures, ensuring the injected structures are consistent with the semantic patterns of the original graph. We model the injection attack as a sequential decision process, which is optimized by a reinforcement learning agent. Experiments on multiple standard datasets demonstrate that the JANUS framework significantly outperforms existing methods in terms of both attack effectiveness and stealthiness. Our code and scripts are available at <https://github.com/jiahaozhang121/JANUS-WWW2026>.

CCS Concepts

• **Security and privacy** → **Machine learning security**; • **Computing methodologies** → *Neural networks*.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792668>

Keywords

Graph Neural Networks, Adversarial Attacks, Node Injection Attacks, Stealthy Attacks

ACM Reference Format:

Jiahao Zhang, Xiaobing Pei, Zhaokun Zhong, Wenqiang Hao, and Zhenghao Tang. 2026. JANUS: A Dual-Constraint Generative Framework for Stealthy Node Injection Attacks. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792668>

1 Introduction

Graph Neural Networks (GNNs) [20, 40] have achieved great success in numerous fields such as node classification [19, 27], graph classification [18, 24], recommendation systems [13, 32], and bioinformatics [9, 29]. However, with the widespread deployment of GNNs in security-sensitive domains like finance, social media, and critical infrastructure networks, their security issues have become increasingly prominent [14]. A large number of studies have shown that GNNs are also vulnerable to adversarial attacks [8, 43]. Attackers can significantly degrade the performance of GNN models, or even induce them to produce specific erroneous outputs, by modifying the topological structure of graphs [15, 39] or injecting maliciously designed nodes into graphs [34, 46], which may lead to severe consequences in practical applications [22, 45].

Node injection is a realistic threat model [42] in many web-centric graph applications, such as e-commerce recommendation systems [13, 32], where new entities can be created with low friction while existing graph data cannot be directly overwritten [34]. In such systems, adversaries can append nodes and establish links to influence downstream decisions, for instance, manipulating item rankings [46]. Meanwhile, modern pipelines often include anomaly screening mechanisms [10, 44], so injected content that is statistically or structurally abnormal will be flagged and filtered before it can take effect [6]. This motivates us to study *high-stealth* node injection attacks [3]—defined as attacks capable of bypassing these detection filters—by remaining plausible at both the feature level

and the structural/semantic level, rather than only optimizing attack success in isolation. Ultimately, exposing such vulnerabilities serves as a critical red-teaming step to reveal the blind spots in current trust systems, thereby facilitating the development of more robust graph learning pipelines for web-scale applications.

Although some progress has been made in the research on improving the stealthiness of injection attacks, there are generally two common limitations:

First, the modeling of stealthiness often relies on the indirect mimicry of heuristic proxy metrics, rather than on fundamental distribution alignment. Many methods attempt to achieve stealthiness by maintaining certain specific attributes of the graph. HAO [5] aims to imitate the proxy metric of homophily, and GANI [11] generates features by sampling the degree of the original graph and based on the statistical data of the target category. These methods only consider the superficial silhouette of the real data distribution. And G-NIA [35], which uses the combination of original node features and scales node attributes to a preset range, is a heuristic post-processing. These methods all lack an end-to-end differentiable constraint that is directly aligned with the real data distribution.

Second, stealthiness constraints are often unidimensional, generally lacking a systematic consideration from the local to the global, which leads to the problem of local myopia. Most existing methods focus on the local environment of the injected node. For example, CANA [36] improves local stealthiness by aligning the distribution of the ego-network of the injected node; while G²A2C [17] only constrains the injected features by limiting the feature budget. TDGIA [47] attacks the GNN model by using the topological defect edge selection strategy with the first-order neighborhood information of the graph. They all lack a constraint that ensures, at the architectural level, that multiple injection behaviors conform to the potential syntax rules of the graph at the global level, so that multiple locally normal injections may still accumulate into a perceptible global structural anomaly.

To overcome the aforementioned limitations, we propose a dual-constraint stealthy node injection framework, called **Joint Alignment of Nodal and Universal Structures (JANUS)**. We reframe the attack as a generative modeling problem of learning the distribution of original graph data. The core is an innovative dual stealthiness constraint mechanism, which systematically solves the dual challenges of local authenticity and global consistency:

1. **Local node feature authenticity:** At the node level, to address the limitations of proxy metrics, we propose a local feature manifold alignment strategy. By introducing the Optimal Transport (OT) [30] theory, we directly measure and align the original feature distribution, and minimize the transport cost between the empirical distribution of the features of the injected nodes and the empirical distribution of the features of the benign nodes sampled from the graph. Geometrically, this ensures that the injected features are a credible sample on the local feature manifold in a geometric sense, fundamentally improving their naturalness in the feature space.

2. **Global graph attribute consistency:** To address the myopic problem of local optimization, we introduce a structure generation strategy based on controllable semantics. By extending the core idea of latent factor disentanglement in InfoGAN [4], we introduce a set of structured latent codes to control the generation process. By maximizing the mutual information between these latent codes and

the generation results, we force the generator to learn the potential, high-level structural patterns and semantic rules in the original graph data.

The generation process of the entire attack is modeled as a sequential decision-making problem, which is optimized by a reinforcement learning agent under the guidance of the above-mentioned dual constraints. The main contributions of this paper can be summarized as follows:

- We propose a novel generative attack framework named JANUS. It systematically addresses the stealthiness challenge from two levels of local feature authenticity and global structure consistency for the first time.
- We design an end-to-end reinforcement learning attack framework. Under a unified optimization objective, it collaboratively realizes the learning of attack efficiency and dual stealthiness constraints.
- Through extensive experiments on multiple benchmark graph datasets, we verify that JANUS can achieve superior attack effects while significantly surpassing existing methods in multiple stealthiness indicators.

2 Preliminaries

We denote a graph as $G = (V, E, X)$, where V is the node set, E is the edge set, and $X \in \mathbb{R}^{|V| \times d}$ is the attribute matrix. The adjacency matrix is $A \in \{0, 1\}^{|V| \times |V|}$, where $A_{ij} = 1$ if an edge exists between node v_i and v_j .

2.1 Adversarial Attacks on Graph Neural Networks

Graph adversarial attacks can be summarized as an optimization problem [48]. Let $G = (V, E, X)$ be the original graph, and f_θ be the GNN model. An attacker aims to find a modified graph $\hat{G} = (V', E', X')$ to maximize a certain attack objective function L_{atk} under the constraint of a perturbation budget Δ , formally expressed as:

$$\max_{\hat{G}: \delta(\hat{G}, G) \leq \Delta} L_{\text{atk}}(f_\theta(\hat{G}), Y_{\text{target}}) \quad (1)$$

where $\delta(\cdot, \cdot)$ is a perturbation metric function. According to the stage at which the attack occurs, it can be divided into poisoning attacks that affect model training [48] and evasion attacks that mislead a fixed model during inference [12].

Based on how attackers manipulate graph data, attacks are mainly categorized into two types: graph structure perturbation [15, 39] and node injection attacks [35]. The former achieves the attack goal by modifying existing edges or node features in the graph, and the perturbed graph has the same node set as the original graph G , i.e., $V(\hat{G}) = V(G)$. The latter realizes the attack by adding new nodes V_{inj} controlled by the attacker and corresponding edges to the graph. Thus, $\hat{G}$ is a supergraph of G , satisfying $V(G) \subset V(\hat{G})$ and $E(G) \subseteq E(\hat{G})$.

In addition, depending on the attacker's knowledge of the target model, attacks can be classified into white-box attacks and black-box attacks. In white-box attacks, the attacker has full knowledge of the model and can utilize its gradients; in the more challenging black-box attacks, the attacker has little knowledge of the model's

internal information and usually can only rely on query feedback [17].

This paper focuses on one of the most challenging scenarios: evasion node injection attacks in a black-box environment. Our core goal is to maximize the attack effect on the target node set $T \subseteq V$ by injecting malicious nodes and edges into the original graph G to form a modified graph $\hat{G}$, under the constraint of a limited number of injected nodes and edges. The objective function of this attack can be formally expressed as maximizing the number of misclassified target nodes:

$$\max_{\hat{G}} \sum_{v \in T} \mathcal{I}(f_{\theta^*}(v, \hat{G}) \neq y_v) \quad (2)$$

where $\mathcal{I}(\cdot)$ is the indicator function. The attack actions must be performed within a strict constraint space to ensure stealthiness. These constraints mainly include:

- (1) Operation Type Constraint: The attack is realized only by injecting new nodes V_{inj} and corresponding edges. Therefore, the new graph $\hat{G} = (\hat{A}, \hat{X})$ is a supergraph of the original graph $G = (A, X)$.
- (2) Budget Constraint: To avoid easy detection, the scale of the injection is strictly limited by a Node Budget Δ_n , which is the total number of allowed injected nodes, and an Edge Budget Δ_e , which is the maximum degree of each injected node.

3 Methodology

Traditional Graph Injection Attacks are often modeled as static optimization problems. However, this paradigm fails to capture the dynamic and generative nature of the attack process. True stealthiness requires injected nodes and edges to not only evade detection but be statistically and structurally integrated with the original graph's data distribution.

Based on this, we propose a fundamental shift in perspective: treating high-stealth node injection attacks as a Generative Modeling Problem. Our goal is to learn a generator that can generate new nodes along with their incident edges conditioned on the current graph state, such that the modified graph is indistinguishable from the true data distribution of the original graph at both statistical and structural levels.

As shown in Figure 1, the proposed JANUS redefines node injection attacks as a generative modeling task. It is centered on a Generative Adversarial Network (GAN) framework driven by Reinforcement Learning (RL), consisting of a generator G_θ as an Actor, a discriminator D_ψ for imposing global constraints, and Critic V_ϕ for evaluating state values. The entire process is optimized through RL mechanisms to maximize attack effectiveness.

3.1 Local Stealthiness Strategy

A primary prerequisite for a successful node injection is that the injected node must be indistinguishable in its individual features.

Our goal is to ensure that the feature vector $\mathbf{x}_{inj}$ becomes a truly credible sample from the underlying data manifold formed by the features of real nodes in its neighborhood. In other words, it must geometrically fall into the dense regions of the benign feature point

cloud in the high-dimensional space, thereby achieving camouflage at a fundamental distributional level.

To achieve this, we propose the **Local Feature Manifold Alignment** strategy. The core idea is that a truly stealthy injected node's feature vector should not merely be statistically similar to benign nodes, but should be a credible sample from the manifold itself. To implement this strategy, we introduce the theory of Optimal Transport (OT) [30]. OT and its computed Wasserstein distance measure the minimum ground cost required to move the mass of one probability distribution to another, thereby deeply capturing and leveraging the geometric structure of the distributions. Crucially, unlike metrics such as KL divergence, the Wasserstein distance can still provide smooth and meaningful gradient signals even when the supports of two empirical distributions do not overlap at all, making it an ideal tool for measuring the degree of point-set integration.

To put the Feature Manifold Alignment strategy into practice, we adopt a targeted, localized comparison approach, as we posit that the naturalness of an injected node's features should be judged relative to its local environment.

Constructing the Target Local Manifold. Before initiating an attack on a specific target node, we first sample a local, benign set of real features from its K-hop neighborhood, denoted as $X_{orig} = \{\mathbf{x}_{orig}^1, \dots, \mathbf{x}_{orig}^n\}$. To ensure representativeness, this sampling is weighted by node degrees. This feature set X_{orig} forms our alignment target and is treated as an empirical probability distribution where mass is uniformly distributed across the n samples:

$$P_{orig} = \frac{1}{n} \sum_{j=1}^n \delta_{\mathbf{x}_{orig}^j} \quad (3)$$

where $\delta_{\mathbf{x}_{orig}^j}$ is the Dirac distribution for the feature of a single node.

Measuring Distance to the Manifold. At each step t of the attack, when a new node feature $\mathbf{x}_{inj}^t$ is generated, we treat it as another single-sample empirical distribution, $P_{inj}^t = \delta_{\mathbf{x}_{inj}^t}$. Our goal is to minimize the Wasserstein distance between P_{inj}^t and P_{orig} . The transport plan from P_{inj}^t to P_{orig} can be described by a coupling matrix γ , whose feasible set is:

$$\Gamma(P_{inj}^t, P_{orig}) = \left\{ \gamma \in \mathbb{R}_+^{1 \times n} \mid \sum_{j=1}^n \gamma_j = 1, \gamma_j \geq 0 \right\} \quad (4)$$

where γ_j represents the proportion of mass transported from $\mathbf{x}_{inj}^t$ to $\mathbf{x}_{orig}^j$. The local stealthiness loss for a single injected node is then defined as the squared Wasserstein-2 distance [30]:

$$\mathcal{L}_{OT}(\mathbf{x}_{inj}^t, X_{orig}) = \min_{\gamma \in \Gamma(P_{inj}^t, P_{orig})} \sum_{j=1}^n \gamma_j \|\mathbf{x}_{inj}^t - \mathbf{x}_{orig}^j\|_2^2 \quad (5)$$

where $\|\cdot\|_2$ denotes the Euclidean norm. In practice, to achieve differentiability, we use the Sinkhorn algorithm [7], which adds an entropy regularization term to the objective. In a complete attack sequence (one episode), we average the OT loss over all T injected nodes to get the final loss term that guides the generator:

$$\mathcal{L}_{OT}^{total} = \frac{1}{T} \sum_{t=1}^T \mathcal{L}_{OT}(\mathbf{x}_{inj}^t, X_{orig}) \quad (6)$$

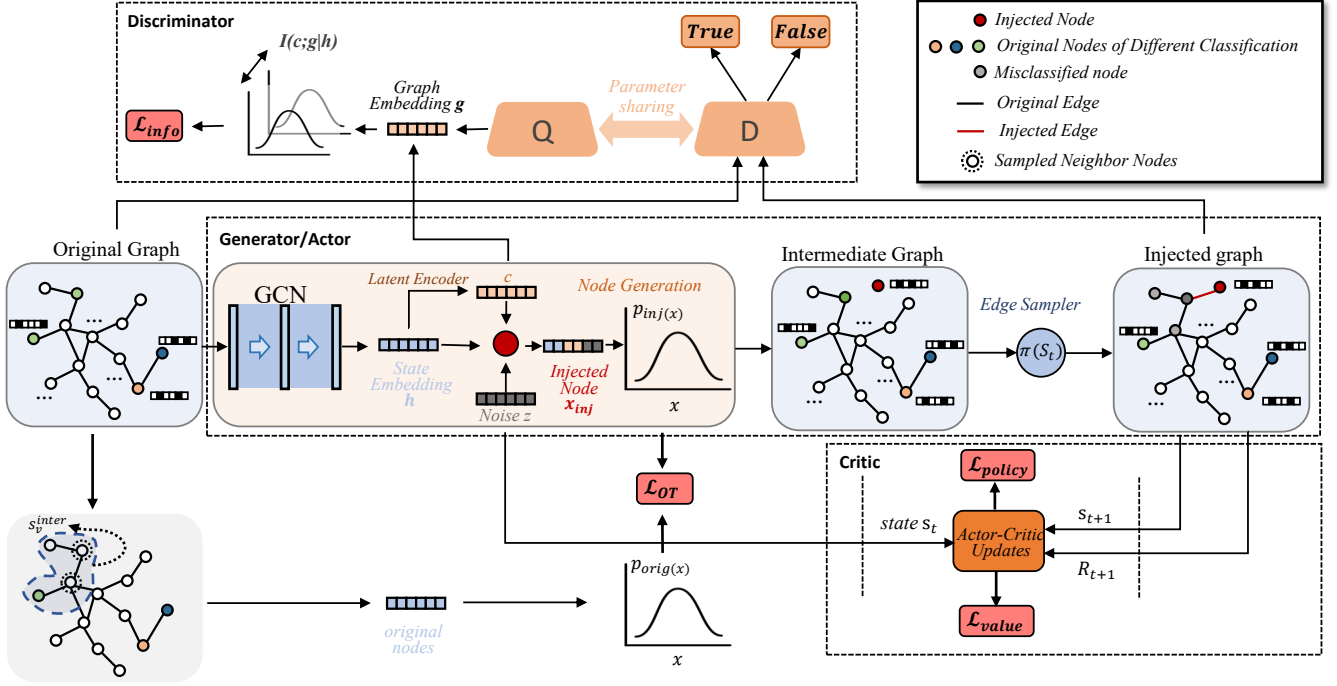


Figure 1: Overview of the JANUS framework. The process operates as a Reinforcement Learning cycle: (1) The Generator (Actor) constructs a malicious node by aligning local features via Optimal Transport and maximizing global semantic consistency via Mutual Information; (2) The Discriminator distinguishes between real and injected patterns to enforce global constraints; (3) The Critic estimates the state value to guide the optimization.

This loss is then added to the RL agent’s total objective function, acting as a penalty for generating unnatural features.

3.2 Global Stealthiness Constraints

Although local feature alignment can camouflage individual nodes, a series of injections that appear locally plausible may accumulate into a perceptible global structural anomaly at the macroscopic level. Our goal is to ensure that the injected perturbations maintain plausibility and consistency on a larger scale of the graph.

To address this challenge, we must move beyond local considerations to ensure that the injected perturbations exhibit plausibility and consistency at a larger graph scale. To this end, we construct a novel global stealthiness strategy, which we term the Global Semantic Structure Alignment strategy. The core idea of this strategy is to maximize the Mutual Information between a small subset of structured latent codes in the generator’s input and the generator’s output. By doing so, we can compel the generator to learn a controllable mapping from semantics, controlled by the latent variables, to structure, the generator’s output, thereby guiding the generation process to follow the high-level patterns learned from the real data.

3.2.1 Discriminator. We introduce a discriminator D_ψ whose goal is to distinguish between real graph data and data forged by the generator. We select a powerful Graph Isomorphism Network (GIN) [41] as the core architecture. The discriminator’s adversarial loss adopts the standard LSGAN [28] loss $\mathcal{L}_{\text{adv}}^{(D)}$. The formula for this

loss is:

$$\begin{aligned}\mathcal{L}_{\text{adv}}^{(D)} &= \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [(D(\mathbf{x}) - 1)^2] \\ &\quad + \frac{1}{2} \mathbb{E}_{\mathbf{h}, \mathbf{c}, \mathbf{z}} [D(G(\mathbf{h}, \mathbf{c}, \mathbf{z}))^2]\end{aligned}\tag{7}$$

3.2.2 Sequential Generator. The generator G_θ of JANUS completes a full node injection action through the sequential operation of two modules:

1. **Node Generator:** To ensure that the generated features $\mathbf{x}_{\text{inj}}$ conform to the feature conventions of the original graph, the Node Generator first uses K-layer Graph Convolutional Layers (GCLs) to perform message propagation on the K-order subgraph of the target node v_i . It then aggregates information via a readout function to construct a context-aware state representation $\mathbf{h}$:

$$\mathbf{h} = \text{Concat} \left(\sum_{v \in S_i} \mathbf{h}_v^{(K)}, \max_{v \in S_i} \mathbf{h}_v^{(K)}, \mathbf{h}_{v_i}^{(K)} \right) \quad (8)$$

where S_i denotes the set of nodes in the K-hop subgraph of v_i , and $\mathbf{h}_v^{(K)}$ is the final K-layer representation for node v . Then, the logits ℓ for the feature distribution are synthesized by a vector that fuses the normalized $\mathbf{h}$, a latent code $\mathbf{c}$, and additional noise $\mathbf{z}$:

$$\ell = \text{MLP}_{\text{node}}(\text{Concat}(\text{Normalize}(\mathbf{h}), \mathbf{c}, \mathbf{z})) \quad (9)$$

Finally, the generator produces the feature vector $\mathbf{x}_{\text{inj}}$ according to the target graph's feature space. This is handled by two distinct mechanisms:

For Discrete Features. : We use the Binary Concrete or Gumbel-Sigmoid reparameterization [26] with a straight-through (ST) estimator [1]. Let ℓ_i be the logit for the i -th feature. We sample logistic noise $\eta_i \sim \text{Logistic}(0, 1)$. The relaxed and hard samples are

$$\tilde{\mathbf{x}}_{\text{inj}}[i] = \sigma\left(\frac{\ell_i + \eta_i}{\tau}\right), \quad \mathbf{x}_{\text{inj}}[i] = \text{round}(\tilde{\mathbf{x}}_{\text{inj}}[i]), \quad (10)$$

where $\sigma(\cdot)$ is the sigmoid and $\tau > 0$ is the temperature. Gradients are backpropagated through $\tilde{\mathbf{x}}_{\text{inj}}[i]$.

For Continuous Features. : To generate a continuous feature vector, the logits ℓ are transformed into a mean vector μ_n and a standard deviation vector σ_n using two separate learnable weight matrices, $\mathbf{W}_\mu$ and $\mathbf{W}_\sigma$. The final feature vector is then sampled from the resulting normal distribution:

$$\mathbf{x}_{\text{inj}} \sim \mathcal{N}(\mu_n, \sigma_n^2) \quad (11)$$

where $\mu_n = \ell \cdot \mathbf{W}_\mu$ and $\sigma_n = \ell \cdot \mathbf{W}_\sigma$.

2. Edge Sampler: After generating node features, the Edge Sampler computes a connection probability vector. For each candidate node v_j , a connection score s_j is calculated. This score is based on a composite representation of the new feature $\mathbf{x}_{\text{inj}}$ and up-to-date node embeddings $(\mathbf{h}_{v_j}, \mathbf{h}_{v_i})$, which are obtained from an internal GNN encoder. A bias term is also added:

$$s_j = \text{MLP}_{\text{edge}}\left(\text{Concat}(\mathbf{h}_{v_j}, \mathbf{h}_{v_i}, \mathbf{x}_{\text{inj}})\right) + \alpha \cdot \mathcal{I}_{v_j \in \mathcal{N}(v_i)} \quad (12)$$

where $\mathcal{I}_{v_j \in \mathcal{N}(v_i)}$ is an indicator function and the bias α promotes the formation of structurally more stealthy triangular loops by increasing the connection probability to existing neighbors. Then, these scores are normalized via a softmax function to produce the final probability distribution over all candidate nodes:

$$p(v_j | v_i, \mathbf{x}_{\text{inj}}) = \text{Softmax}_j(s_j) \quad (13)$$

An edge is finally sampled according to this distribution to complete the connection.

3.2.3 Latent Coding. To solve the local myopia problem, we introduce structured latent variables $\mathbf{c} = [\mathbf{c}_{\text{disc}}, \mathbf{c}_{\text{cont}}]$ to model discrete semantic categories and continuous attributes, respectively. The sampling distribution of latent variables is dynamically adjusted by a context-aware latent encoder based on the state representation $\mathbf{h}$:

$$p_{\text{disc}}, \mu_c, \sigma_c = \text{LatentEncoder}(\mathbf{h}) \quad (14)$$

$$\mathbf{c}_{\text{disc}} \sim \text{Categorical}(p_{\text{disc}}), \quad \mathbf{c}_{\text{cont}} \sim \mathcal{N}(\mu_c, \sigma_c^2)$$

To ensure that the generator uses the latent code $\mathbf{c}$ to learn high-order structural semantics of the attack, we maximize the conditional mutual information between $\mathbf{c}$ and a representation of the final generated structure. However, directly computing and optimizing the conditional mutual information $I(\mathbf{c}; \mathbf{g} | \mathbf{h})$ is difficult, as it involves the posterior probability $P(\mathbf{c} | \mathbf{g}, \mathbf{h})$, which is often intractable. To overcome this, we employ a variational inference approach, similar to InfoGAN [4], by introducing an auxiliary network Q to approximate this posterior. Maximizing a computable

variational lower bound of the mutual information thus becomes the tractable proxy objective.

Since our generator G is a sequential policy conditioned on the state representation $\mathbf{h}$, the structured code $\mathbf{c}$, and unstructured noise $\mathbf{z}$, its final output is a modified subgraph. We obtain a graph-level embedding, denoted as $\mathbf{g}$, by inputting this modified subgraph into the encoder component of our discriminator network D_ψ . The mutual information objective is then optimized via the following loss:

$$\mathcal{L}_{\text{info}} = -\mathbb{E}_{\mathbf{h}, \mathbf{c}, \mathbf{z}} [\log Q(\mathbf{c} | \mathbf{g}, \mathbf{h})] \quad (15)$$

where the auxiliary network Q is trained to recover the latent code $\mathbf{c}$ from the graph-level embedding $\mathbf{g}$ and the original context $\mathbf{h}$. This network Q is integrated into the main discriminator structure D_ψ and shares most of its parameters. For discrete latent codes $\mathbf{c}_{\text{disc}}$, cross-entropy loss is used here. For continuous latent codes $\mathbf{c}_{\text{cont}}$, the negative log-likelihood under the Gaussian distribution predicted by Q is minimized.

Finally, the total loss of the discriminator is the sum of the adversarial loss and the information loss:

$$\mathcal{L}_D^{\text{total}} = \mathcal{L}_{\text{adv}}^{(D)} + \mathcal{L}_{\text{info}} \quad (16)$$

3.3 RL-Driven Attack Optimization

3.3.1 MDP Formulation. To address challenges such as the non-differentiability of black-box objectives and the sequential nature of the decision-making process, we reframe the attack process from a static optimization problem to a formal Markov Decision Process (MDP). This paradigm is necessary because a successful attack comprises a series of interdependent decisions: each injected node or edge alters the graph's state, profoundly influencing the effectiveness of subsequent actions. The MDP provides the rigorous mathematical language to model this dynamic evolution of state and temporal dependency between decisions. We formally define this process as an MDP tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

Within this framework, the core components are carefully designed. The state $\mathbf{s}_t$ is defined as a complete representation containing all necessary information at a decision step, primarily the currently modified graph structure and node features, along with information about the attack target. This definition ensures the Markov property is satisfied. The action $\mathbf{a}_t$ is designed as a composite operation, consisting of two sub-actions: node generation and edge sampling. This decomposition logically separates the attacker's decisions into creating a malicious entity and strategically placing it within the graph.

The reward function serves as the bridge between the non-differentiable attack objective and the agent's learning process. Our reward is purely attack-driven. Let the victim's classification loss on a target node v in graph G be the cross-entropy loss:

$$\ell(v, G) = \text{CE}(y_v, f_{\theta^*}(v, G))$$

The immediate reward r_t for an action at step t is calculated as the increase in this loss, supplemented by a discrete bonus β for a successful misclassification:

$$r_t = \ell(v_{\text{target}}, G_{t+1}) - \ell(v_{\text{target}}, G_t) + \beta \mathcal{I}(f_{\theta^*}(v_{\text{target}}, G_{t+1}) \neq y_{v_{\text{target}}}) \quad (17)$$

where $\mathcal{I}$ is the indicator function. The process is guided by a discount factor $\gamma \in [0, 1]$ to balance immediate versus future rewards.

A key design choice is the separation of this external environmental feedback (reward) from the agent’s internal constraints (the stealthiness requirements). Instead of being formulated as negative rewards, the stealthiness penalties are incorporated directly into the agent’s loss function.

We adopt an Actor-Critic architecture to solve this MDP. The architecture consists of two parts: the **Actor** network, a policy network π_θ , which is also our generator, responsible for outputting an action $\mathbf{a}_t$ based on the current state $\mathbf{s}_t$; and the **Critic** network, a value network V_ϕ that estimates the state value $V_\phi(\mathbf{s}_t)$.

3.3.2 Unified Training Objective. The training of the generator (Actor) involves a multi-objective optimization driven by a unified loss function, $\mathcal{L}_G^{\text{total}}$. To compute the policy and value losses, we first define the **discounted cumulative return** from step t as R_t :

$$R_t = \sum_{k=t}^T \gamma^{k-t} r_k \quad (18)$$

where T is the terminal step of an episode.

The primary component for the generator is the policy loss, which encourages actions that maximize the attack success rate via the advantage function $A_t = R_t - V_\phi(\mathbf{s}_t)$. To ensure global stealthiness, a generator’s adversarial loss $\mathcal{L}_{\text{adv}}^{(G)}$ is incorporated to train the generator to fool the discriminator. These terms, along with an information loss $\mathcal{L}_{\text{info}}$ and a local stealthiness loss $\mathcal{L}_{\text{OT}}$, are combined to form the generator’s total training objective:

$$\mathcal{L}_G^{\text{total}} = \mathcal{L}_{\text{policy}} + \mathcal{L}_{\text{adv}}^{(G)} + \mathcal{L}_{\text{info}} + \lambda_{\text{OT}} \mathcal{L}_{\text{OT}} \quad (19)$$

where the policy loss is defined as $\mathcal{L}_{\text{policy}} = \sum -\log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) A_t$.

Concurrently, the Critic network is optimized by minimizing a separate **value loss** to make its prediction $V_\phi(\mathbf{s}_t)$ closer to the actual return R_t . This loss is calculated using the Smooth L1 loss function:

$$\mathcal{L}_{\text{value}} = \sum \text{SmoothL1Loss}(V_\phi(\mathbf{s}_t), R_t) \quad (20)$$

The training alternates between updating the generator, the discriminator, and the critic. The detailed training procedure is provided in **Algorithm 1** in **Appendix B**.

4 Experiments

In this section, our experimental design aims to answer the following core research questions: (RQ1) How does the attack success rate of JANUS compare to existing baselines? (RQ2) Can JANUS maintain strong attack capability when facing mainstream graph defense mechanisms? (RQ3) What is the stealthiness performance of JANUS’s injected nodes from both quantitative and visual perspectives? (RQ4) Do the core components in JANUS make essential contributions to its final performance?

4.1 Experimental Setup

Datasets. We conducted experiments on eight well-recognized benchmark datasets, including citation networks Cora, Citeseer, Pubmed [31], and Cora-ML [2]; the co-purchase network Amazon Photo [33]; the Bayesian network UAI [38]; and the Open Graph Benchmark OGB-Products [16]. These datasets cover multiple domains and feature spaces, which are sufficient to fully verify our method’s performance. For UAI and Cora-ML, we adopted a random

split of 20%/20%/60% for training, validation, and testing. For other datasets, we used the same subgraphs and data splits as G²A2C [17] to ensure a fair comparison. Detailed statistics are shown in Table 1.

Table 1: Statistical Information of Experimental Datasets.

Dataset	Node	Edge	Class	Dim.
Datasets with Discrete Feature Space				
Cora	2,708	5,429	7	1,433
Citeseer	3,327	4,732	6	3,703
Am. Photo	7,650	119,043	8	745
Uai	3,067	28,311	19	4,973
Datasets with Continuous Feature Space				
Pubmed	19,717	44,338	3	500
Wiki. CS	11,701	216,123	10	300
OGB-Prod.	10,494	77,656	35	100
Cora_ml	2,995	4,208	7	2,879

Baselines. To comprehensively evaluate the performance of JANUS, we compare it with a series of state-of-the-art node injection attack methods. These baselines include reinforcement learning-based black-box methods, such as NIPA [34] and G²A2C [17], as well as proxy gradient-based methods like G-NIA [35], TDGIA [47], and AFGSM [37]. For methods that require a white-box setting, we uniformly trained a two-layer GCN as their proxy model. In addition, we introduced camouflage enhancement frameworks HAO [5] and CANA [36] combined with TDGIA as stronger stealthy attack baselines.

Implementation Details. For all baselines, we adopt their default parameter settings or use implementations from the DeepRobust library [23]. For our proposed JANUS framework, we use the Adam optimizer with learning rates of 1e-4 for the generator/critic and 1e-5 for the discriminator. The GCN hidden dimension is 128. The hyperparameter λ_{OT} is selected via grid search in the range [0.1, 1] with a step of 0.1. The dimension of discrete latent codes is the same as the number of dataset categories, and the dimension of continuous latent codes is set to 15. Training is conducted for a maximum of 10,000 epochs with an early stopping mechanism with a patience of 15. All experiments are conducted on a server equipped with an NVIDIA A800 GPU and an Intel Xeon Gold 6348 CPU; each experiment is repeated ten times and the average results are reported.

4.2 Analysis of Attack Effectiveness

To evaluate the attack effectiveness of JANUS, we first conducted attacks on a two-layer GCN victim model under the highly challenging single-node and single-edge injection setting. As reported in Table 2, JANUS achieves the highest misclassification rate across all datasets, significantly outperforming all baseline methods. These improvements are statistically significant across all datasets ($p < 0.01$). On the widely used Citeseer and Cora_ml benchmarks, for instance, JANUS achieves misclassification rates of 66.9% and 79.3%

Table 2: Misclassification rates (%) on two-layer GCN under single-node and single-edge injection attacks. The best results are in bold and the second-best are underlined. We report the mean and standard deviation over 10 runs with different seeds. The improvements of our method over all baselines are verified to be statistically significant via paired t-tests ($p < 0.01$).

Attacker	Discrete Feature Space				Continuous Feature Space			
	Cora	Citeseer	Uai	Am. Photo	OGB-Prod.	Cora_ml	Pubmed	Wiki CS
Clean	19.1	25.1	27.7	17.8	23.2	14.4	21.9	18.3
NIPA	19.7 $\pm$ 0.2	25.2 $\pm$ 0.1	27.9 $\pm$ 0.3	17.8 $\pm$ 0.	24.1 $\pm$ 0.3	15.1 $\pm$ 0.2	21.9 $\pm$ 0.	19.2 $\pm$ 0.4
AFGSM	26.1 $\pm$ 3.2	39.9 $\pm$ 3.5	30.6 $\pm$ 2.1	32.3 $\pm$ 1.9	75.2 $\pm$ 0.8	63.5 $\pm$ 1.1	66.0 $\pm$ 1.3	75.2 $\pm$ 0.9
G-NIA	24.5 $\pm$ 2.8	40.5 $\pm$ 3.0	32.3 $\pm$ 2.5	25.0 $\pm$ 1.8	<u>97.1$\pm$0.5</u>	69.3 $\pm$ 1.1	68.1 $\pm$ 1.0	79.3 $\pm$ 1.5
TDGIA	31.2 $\pm$ 2.5	44.2 $\pm$ 2.8	32.5 $\pm$ 2.4	<u>34.1$\pm$1.3</u>	<u>95.3$\pm$0.4</u>	68.7 $\pm$ 1.2	70.9 $\pm$ 0.7	84.2 $\pm$ 1.1
G ² A2C	39.1 $\pm$ 2.9	<u>50.3$\pm$3.2</u>	<u>34.3$\pm$2.2</u>	<u>33.3$\pm$1.5</u>	96.0 $\pm$ 0.4	<u>72.6$\pm$1.3</u>	<u>73.4$\pm$0.9</u>	<u>86.1$\pm$0.9</u>
JANUS (ours)	60.7$\pm$3.1	66.9$\pm$3.1	46.4$\pm$1.6	41.0$\pm$1.1	98.6$\pm$0.2	79.3$\pm$0.9	89.5$\pm$1.2	91.8$\pm$1.2
Avg. $\uparrow$	21.6	16.6	12.1	6.9	1.5	6.7	16.1	5.7

respectively, far exceeding all baseline methods. This superior performance, even under the strictest budget, demonstrates a key insight of our work: stealthiness is not a trade-off against efficacy but a direct enabler of it. The dual-stealthiness mechanism compels JANUS to discover fundamentally more deceptive and effective attack vectors within the natural data manifold, thus comprehensively answering RQ1.

4.3 Robustness Against Defenses

To evaluate the robustness of JANUS in realistic adversarial environments, we test its effectiveness against two mainstream defense models: GNNGuard [44] and FLAG [21]. In a large-scale attack on Citeseer, we inject 1000 malicious nodes, each with 2 edges, while on OGB-Products, we inject 2099 nodes, each with 7 edges. As shown in Table 3, JANUS demonstrates superior robustness, consistently outperforming baselines. This advantage stems from producing attack patterns fundamentally harder for defenses to neutralize. By ensuring both local feature authenticity and global structural consistency, its natural features evade feature-level scrutiny, while its globally coherent structure bypasses topological defense mechanisms like GNNGuard’s attention, thus providing a clear answer to RQ2.

Table 3: Misclassification Rates (%) under Defense Models.

Attacker	Backbone	Citeseer	OGB-Prod.
TDGIA	FLAG	43.7	75.7
	GNNGuard	45.8	80.5
TDGIA +CANA	FLAG	40.0	82.2
	GNNGuard	48.9	84.1
TDGIA +HAO	FLAG	47.9	87.3
	GNNGuard	49.6	90.9
JANUS	FLAG	50.5	98.1
	GNNGuard	60.4	93.2

4.4 Stealthiness Evaluation

We first evaluate stealthiness using quantitative metrics to answer RQ3. We adopt two common metrics where lower is better: Closest Attribute Distance (CAD) [36], which measures feature similarity to the nearest original node, and Smoothness [10], which measures feature consistency with connected neighbors. CAD measures feature-level plausibility; a low CAD score, as achieved by JANUS, indicates that the injected features are not outliers but closely resemble existing nodes, making them difficult for feature-based anomaly detectors to flag and directly validating our local feature manifold alignment strategy. In parallel, Smoothness evaluates local structural consistency; a low smoothness value demonstrates that an injected node fits seamlessly into its new local environment, showcasing the synergy between our feature generator and intelligent edge sampling mechanism. The results, presented in Figure 2, show that JANUS consistently achieves the best performance on the OGB-Products dataset. It outperforms all baselines across both metrics, establishing its state-of-the-art (SOTA) stealthiness. Achieving top performance on both is significant, as it confirms that JANUS successfully masters the dual challenges of generating individually realistic features while also integrating them into the graph in a structurally coherent way.

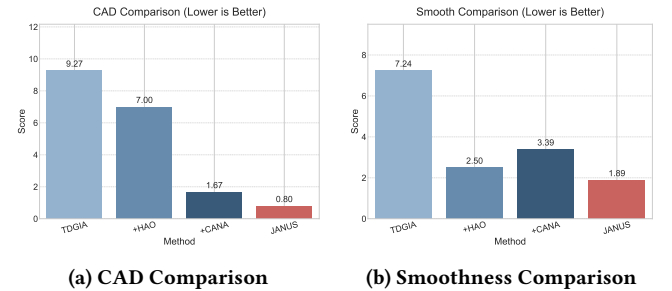


Figure 2: Quantitative stealth metrics (↓ lower is better) on the OGB-Products dataset.

To provide more intuitive evidence, we present a t-SNE visualization [25] in Figure 3 that offers two complementary insights. First,

the IForest detection AUCs reported in the figure’s captions serve to validate our local feature manifold alignment. The baseline’s high AUC of 0.90 indicates its features are detectable, whereas the near-random 0.48 AUC for JANUS proves its superior feature-level stealth. Second, the visual distribution of the nodes validates our global semantic structure alignment. As shown in the visualization, the baseline’s injected nodes form distinct clusters. In stark contrast, JANUS’s nodes are uniformly distributed and seamlessly integrated with the original node distribution. Crucially, JANUS achieves this comprehensive, SOTA stealth while maintaining its superior attack success rate (Table 2), thus establishing a more advanced Pareto front.

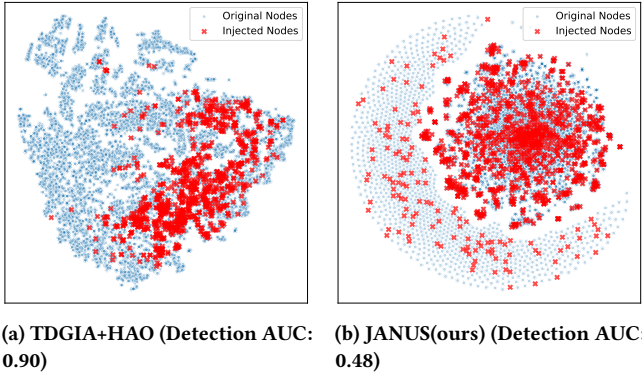


Figure 3: t-SNE visualization of injected nodes (red) versus original nodes (blue) on the OGB-Products dataset.

4.5 Ablation Study

Table 4: Attack success rate (%) of JANUS variants on Citeseer.

Variant	GCN	GNNGuard	FLAG
JANUS	66.9	56.7	46.8
JANUS w/o local	58.3	49.7	43.2
JANUS w/o global	52.6	47.9	40.7
RL-only	43.2	39.2	36.3

To verify the effectiveness of each core component in JANUS (RQ4), we designed three ablation variants: **JANUS w/o local**, **JANUS w/o global**, and a pure **RL-only** baseline. The results in Table 4 demonstrate the distinct contribution of each component.

The local OT alignment is responsible for ensuring feature-level authenticity, making an injected node’s attributes statistically similar to its neighbors. Its removal (**JANUS w/o local**) leads to a notable drop in performance, particularly against the robust GNNGuard defense, confirming that generating natural features is crucial for bypassing defenses that scrutinize feature patterns.

The global stealthiness module is designed to solve the local myopia problem by forcing the generated subgraphs to adhere to the broader structural and semantic rules of the entire graph. Removing this module (**JANUS w/o global**) causes the most severe

performance degradation. This underscores its critical role: without global consistency, even a series of locally plausible injections accumulate into a detectable, anomalous pattern at the macroscopic level, rendering the attack far less effective.

Finally, the poor performance of the **RL-only** variant, which lacks any explicit stealth constraints, serves as a crucial baseline. It confirms that the attack’s success is not merely a result of the reinforcement learning framework, but is fundamentally driven by the sophisticated dual-stealthiness guidance. The superior performance of the full JANUS model confirms both constraints are indispensable and synergistic, thus answering RQ4.

5 Related Work

Adversarial attacks on GNNs are primarily categorized into two paradigms: Graph Manipulation Attacks (GMA) and Graph Injection Attacks (GIA). GMA methods [48] perturb existing edges or features but are often impractical due to the requirement of direct data modification. In contrast, GIAs [34], which only add new nodes and connections, represent a more feasible threat model.

However, existing GIA methods exhibit significant limitations. A prominent line of work, including G-NIA [35] and TDGIA [47], relies on gradients from surrogate models. The effectiveness of these methods degrades significantly when the surrogate model diverges from the true victim architecture. Furthermore, their focus on local feature simulation without ensuring global consistency often leads to a local myopia problem. More recently, G²A2C [17] introduced a RL approach to operate in a gradient-free, black-box setting. Nevertheless, it still relies on heuristic constraints for stealthiness and does not explicitly enforce the naturalness of the global graph semantics.

To address these deficiencies, this paper proposes JANUS, which integrates optimal transport for local feature manifold alignment and mutual information maximization for global structural semantics into a unified RL framework to simultaneously enhance attack effectiveness and stealthiness.

6 Conclusion

In this work, we study the challenging problem of stealthy black-box node injection attacks against GNNs. To address this problem, we propose JANUS, a generative attack framework centered on a novel dual-stealthiness constraint mechanism. We reframe the attack as a generative modeling problem and model it as a sequential decision process, optimized by a reinforcement learning agent. This agent is guided by two key principles: local feature manifold alignment via optimal transport and global structural consistency via mutual information maximization, ensuring both feature and structural naturalness. Through comprehensive experiments, we demonstrate that JANUS significantly outperforms existing SOTA methods in both attack success rate and multiple stealthiness metrics. Furthermore, it maintains high effectiveness even when facing advanced defense mechanisms, proving the superiority of its holistic approach to stealth. Ultimately, JANUS unveils latent vulnerabilities within current graph learning pipelines, establishing a rigorous benchmark to catalyze the development of resilient defense mechanisms for security-critical applications.

References

- [1] Yoshua Bengio. 2013. Estimating or propagating gradients through stochastic neurons. *arXiv preprint arXiv:1305.2982* (2013).
- [2] A. Bojchevski and S. Günnemann. 2017. Deep Gaussian Embedding of Graphs: Unsupervised Inductive Learning via Ranking. *arXiv:1707.03815*
- [3] T. Cai, Y. Jiang, M. Li, L. Bai, C. Huang, and Y. Wang. 2024. HyperNear: Unnoticeable Node Injection Attacks on Hypergraph Neural Networks. In *Forty-second International Conference on Machine Learning*.
- [4] X. Chen, Y. Duan, R. Houthoofd, J. Schulman, I. Sutskever, and P. Abbeel. 2016. InfoGAN: Interpretable Representation Learning by Information Maximizing Generative Adversarial Nets. *Advances in Neural Information Processing Systems* 29 (2016).
- [5] Y. Chen, H. Yang, Y. Zhang, K. Ma, T. Liu, B. Han, and J. Cheng. 2022. Understanding and Improving Graph Injection Attack by Promoting Unnoticeability. *arXiv:2202.08057*
- [6] Y. Chen, Z. Ye, Z. Wang, and H. Zhao. 2024. Imperceptible Graph Injection Attack on Graph Neural Networks. *Complex & Intelligent Systems* 10, 1 (2024), 869–883.
- [7] M. Cuturi. 2013. Sinkhorn Distances: Lightspeed Computation of Optimal Transport. *Advances in Neural Information Processing Systems* 26 (2013).
- [8] E. Dai, T. Zhao, H. Zhu, J. Xu, Z. Guo, H. Liu, and S. Wang. 2024. A Comprehensive Survey on Trustworthy Graph Neural Networks: Privacy, Robustness, Fairness, and Explainability. *Machine Intelligence Research* 21, 6 (2024), 1011–1061.
- [9] G. Dong, M. Tang, Z. Wang, J. Gao, S. Guo, L. Cai, and M. Boukhechba. 2023. Graph Neural Networks in IoT: A Survey. *ACM Transactions on Sensor Networks* 19, 2 (2023), 1–50.
- [10] X. Dong, X. Zhang, and S. Wang. 2023. Rayleigh Quotient Graph Neural Networks for Graph-Level Anomaly Detection. *arXiv:2310.02861*
- [11] J. Fang, H. Wen, J. Wu, Q. Xuan, Z. Zheng, and C. K. Tse. 2024. GANI: Global Attacks on Graph Neural Networks via Imperceptible Node Injections. *IEEE Transactions on Computational Social Systems* 11, 4 (2024), 5374–5387.
- [12] W. Feng, B. Wu, T. Zhang, Y. Zhang, and Y. Zhang. 2021. Meta-Attack: Class-Agnostic and Model-Agnostic Physical Adversarial Attack. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 7787–7796.
- [13] C. Gao, Y. Zheng, N. Li, Y. Li, Y. Qin, J. Piao, and Y. Li. 2023. A Survey of Graph Neural Networks for Recommender Systems: Challenges, Methods, and Directions. *ACM Transactions on Recommender Systems* 1, 1 (2023), 1–51.
- [14] F. Guan, T. Zhu, W. Zhou, and K. R. Choo. 2024. Graph Neural Networks: A Survey on the Links Between Privacy and Security. *Artificial Intelligence Review* 57, 2 (2024), 40.
- [15] C. Hu, R. Yu, B. Zeng, Y. Zhan, Y. Fu, Q. Zhang, and H. Shi. 2023. Hyperattack: Multi-gradient-guided white-box adversarial structure attack of hypergraph neural networks. *arXiv:2302.12407*
- [16] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *Advances in Neural Information Processing Systems* 33 (2020), 22118–22133.
- [17] M. Ju, Y. Fan, C. Zhang, and Y. Ye. 2023. Let Graph Be the Go Board: Gradient-Free Node Injection Attack for Graph Neural Networks via Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 4383–4390.
- [18] B. Khemani, S. Patil, K. Kotecha, and S. Tanwar. 2024. A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. *Journal of Big Data* 11, 1 (2024), 18.
- [19] S. Khoshraftar and A. An. 2024. A Survey on Graph Representation Learning Methods. *ACM Transactions on Intelligent Systems and Technology* 15, 1 (2024), 1–55.
- [20] T. N. Kipf and M. Welling. 2016. Semi-Supervised Classification with Graph Convolutional Networks. *arXiv:1609.02907*
- [21] K. Kong, G. Li, M. Ding, Z. Wu, C. Zhu, B. Ghanem, and T. Goldstein. 2020. FLAG: Adversarial Data Augmentation for Graph Neural Networks. *arXiv:2010.09891*
- [22] D. Li, H. Wu, M. Xie, X. Wu, Z. Wu, and W. Zhang. 2024. Talos: A More Effective and Efficient Adversarial Defense for GNN Models Based on the Global Homophily of Graphs. *arXiv:2406.03833*
- [23] Yaxin Li, Wei Jin, Han Xu, and Jiliang Tang. 2020. Deeprobust: A pytorch library for adversarial attacks and defenses. *arXiv preprint arXiv:2005.06149* (2020).
- [24] Xingyu Liu, Juan Chen, and Quan Wen. 2023. A survey on graph classification and link prediction based on gnn. *arXiv preprint arXiv:2307.00865* (2023).
- [25] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, Nov (2008), 2579–2605.
- [26] Chris J Maddison, Andriy Mnih, and Yee Whye Teh. 2016. The concrete distribution: A continuous relaxation of discrete random variables. *arXiv preprint arXiv:1611.00712* (2016).
- [27] A. M. Mahmoud, A. S. Desuky, H. Fathy, and H. Abdeldaim. 2024. An Overview and Evaluation on Graph Neural Networks for Node Classification. *International Journal of Theoretical and Applied Research* 3, 1 (2024), 379–386.
- [28] X. Mao, Q. Li, H. Xie, R. Y. Lau, Z. Wang, and S. Paul Smolley. 2017. Least Squares Generative Adversarial Networks. In *Proceedings of the IEEE International Conference on Computer Vision*. 2794–2802.
- [29] S. G. Paul, A. Saha, M. Z. Hasan, S. R. H. Noori, and A. Moustafa. 2024. A Systematic Review of Graph Neural Network in Healthcare-Based Applications: Recent Advances, Trends, and Future Directions. *IEEE Access* 12 (2024), 15145–15170.
- [30] G. Peyré and M. Cuturi. 2019. Computational Optimal Transport: With Applications to Data Science. *Foundations and Trends in Machine Learning* 11, 5–6 (2019), 355–607.
- [31] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (2008), 93–93.
- [32] K. Sharma, Y. C. Lee, S. Nambi, A. Salian, S. Shah, S. W. Kim, and S. Kumar. 2024. A Survey of Graph Neural Networks for Social Recommender Systems. *Comput. Surveys* 56, 10 (2024), 1–34.
- [33] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann. 2018. Pitfalls of Graph Neural Network Evaluation. *arXiv:1811.05868*
- [34] Y. Sun, S. Wang, X. Tang, T. Y. Hsieh, and V. Honavar. 2020. Adversarial Attacks on Graph Neural Networks via Node Injections: A Hierarchical Reinforcement Learning Approach. In *Proceedings of The Web Conference 2020*. 673–683.
- [35] S. Tao, Q. Cao, H. Shen, J. Huang, Y. Wu, and X. Cheng. 2021. Single Node Injection Attack Against Graph Neural Networks. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 1794–1803.
- [36] S. Tao, Q. Cao, H. Shen, Y. Wu, L. Hou, F. Sun, and X. Cheng. 2023. Adversarial Camouflage for Node Injection Attack on Graphs. *Information Sciences* 649 (2023), 119611.
- [37] Jiong Wang, Minnan Luo, Fnu Suya, Jundong Li, Zijiang Yang, and Qinghua Zheng. 2020. Scalable Attack on Graph Data by Injecting Vicious Nodes. *Data Mining and Knowledge Discovery* 34, 5 (2020), 1363–1389.
- [38] W. Wang, X. Liu, P. Jiao, X. Chen, and D. Jin. 2018. A Unified Weakly Supervised Framework for Community Detection and Semantic Matching. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer International Publishing, 218–230.
- [39] T. Wu, C. Cui, X. Xian, S. Qiao, C. Wang, L. Yuan, and S. Yu. 2024. Explainable AI Security: Exploring Robustness of Graph Neural Networks to Adversarial Attacks. *arXiv:2406.13920*
- [40] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu. 2020. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (2020), 4–24.
- [41] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. 2018. How Powerful are Graph Neural Networks? *arXiv:1810.00826*
- [42] O. Zari, J. Parra-Arnau, A. Unsal, and M. Önen. 2024. Node Injection Link Stealing Attack. In *International Conference on Privacy in Statistical Databases*. Springer Nature Switzerland, 358–373.
- [43] B. Zhang, Y. Dong, C. Chen, Y. Zhu, M. Luo, and J. Li. 2023. Adversarial Attacks on Fairness of Graph Neural Networks. *arXiv:2310.13822*
- [44] X. Zhang and M. Zitnik. 2020. GNNGuard: Defending Graph Neural Networks Against Adversarial Attacks. *Advances in Neural Information Processing Systems* 33 (2020), 9263–9275.
- [45] K. Zhao, Q. Kang, Y. Song, R. She, S. Wang, and W. P. Tay. 2023. Adversarial Robustness in Graph Neural Networks: A Hamiltonian Approach. *Advances in Neural Information Processing Systems* 36 (2023), 3338–3361.
- [46] P. Zhu, Z. Pan, K. Tang, X. Cui, J. Wang, and Q. Xuan. 2024. Node Injection Attack Based on Label Propagation Against Graph Neural Network. *IEEE Transactions on Computational Social Systems* (2024).
- [47] X. Zou, Q. Zheng, Y. Dong, X. Guan, E. Kharlamov, J. Lu, and J. Tang. 2021. TDGIA: Effective Injection Attacks on Graph Neural Networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2461–2471.
- [48] D. Zügner, A. Akbarnejad, and S. Günnemann. 2018. Adversarial Attacks on Neural Networks for Graph Data. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2847–2856.

A Mathematical Derivations

A.1 Mutual Information Maximization and Q-Network Derivation

To ensure that the latent code $\mathbf{c}$ meaningfully controls the generated structure in a context-aware manner (i.e., conditioned on $\mathbf{h}$), we maximize the **Conditional Mutual Information (CMI)** $I(\mathbf{c}; \mathbf{g}|\mathbf{h})$ between the code and the generated structure’s embedding $\mathbf{g}$. As direct CMI calculation is intractable, we introduce an auxiliary network, $Q(\mathbf{c}|\mathbf{g}, \mathbf{h})$, to approximate the true posterior $P(\mathbf{c}|\mathbf{g}, \mathbf{h})$ and maximize a variational lower bound of the CMI, following the derivation from InfoGAN [4].

Algorithm 1: Training Algorithm of JANUS

```

1: Input: Graph  $G$ , victim model  $M$ , training rounds  $N$ , attack budget  $\Delta$ 
2: Output: Optimal attack policy  $\pi_\theta^*$ 
3: Initialize Generator (Actor)  $\theta$ , Discriminator  $\psi$ , and Critic  $\phi$ .
4: for epoch = 1 to  $N$  do
5:   Get initial state  $s_0$  from the environment.
6:   for  $t = 0$  to  $\Delta - 1$  do
7:     Sample action  $\mathbf{a}_t = (\mathbf{x}_{\text{inj}}, \mathbf{e}_{\text{inj}})$  from policy  $\pi_\theta(\cdot | \mathbf{s}_t)$ .
8:     Calculate local stealthiness cost  $\mathcal{L}_{\text{OT}}$  for  $\mathbf{x}_{\text{inj}}$ .
9:     Execute action  $\mathbf{a}_t$ , get next state  $\mathbf{s}_{t+1}$  and reward  $r_t$ .
10:    Store transition  $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$  in buffer  $B$ .
11:   end for
12:   Update Critic  $\phi$  by minimizing  $\mathcal{L}_{\text{value}}$  on a batch from  $B$ .
13:   Update Generator (Actor)  $\theta$  by minimizing  $\mathcal{L}_G^{\text{total}}$  on a batch from  $B$ .
14:   Update Discriminator  $\psi$  by minimizing  $\mathcal{L}_D^{\text{total}}$  on a batch from  $B$ .
15: end for

```

The CMI can be expressed and bounded as follows:

$$\begin{aligned}
I(\mathbf{c}; \mathbf{g} | \mathbf{h}) &= H(\mathbf{c} | \mathbf{h}) - H(\mathbf{c} | \mathbf{g}, \mathbf{h}) \\
&= H(\mathbf{c} | \mathbf{h}) + \mathbb{E}_{\mathbf{g}, \mathbf{h}} [\mathbb{E}_{\mathbf{c}' \sim P(\mathbf{c} | \mathbf{g}, \mathbf{h})} [\log P(\mathbf{c}' | \mathbf{g}, \mathbf{h})]] \\
&= H(\mathbf{c} | \mathbf{h}) + \mathbb{E}_{\mathbf{g}, \mathbf{h}} [D_{\text{KL}}(P || Q) + \mathbb{E}_{\mathbf{c}' \sim P} [\log Q(\mathbf{c}' | \mathbf{g}, \mathbf{h})]] \\
&\geq H(\mathbf{c} | \mathbf{h}) + \mathbb{E}_{\mathbf{h}, \mathbf{c}, \mathbf{z}} [\log Q(\mathbf{c} | \mathbf{g}, \mathbf{h})]
\end{aligned} \tag{21}$$

where the relationship $\mathbf{g} = G_\theta(\mathbf{h}, \mathbf{c}, \mathbf{z})$ is used to transition the expectation. The inequality holds because the KL divergence is non-negative. While the conditional entropy $H(\mathbf{c} | \mathbf{h})$ is not a constant, in practice, the information game between the generator G and the Q-network is primarily driven by maximizing the expectation term $\mathbb{E}[\log Q(\mathbf{c} | \mathbf{g}, \mathbf{h})]$. We therefore train the Q-network by minimizing its negative, which gives the information loss $\mathcal{L}_{\text{info}}$ used in our main paper:

$$\mathcal{L}_{\text{info}} = -\mathbb{E}_{\mathbf{h}, \mathbf{c}, \mathbf{z}} [\log Q(\mathbf{c} | \mathbf{g}, \mathbf{h})] \tag{22}$$

In practice, minimizing this negative log-likelihood loss is equivalent to training the Q-network to accurately predict the original latent code $\mathbf{c}$ from the generator's output $\mathbf{g}$ and the original context $\mathbf{h}$.

A.2 Entropy-Regularized Optimal Transport

We adopt entropy-regularized OT to obtain a differentiable approximation of the Wasserstein cost. Let $\mathbf{r} \in \Delta^n$ and $\mathbf{c} \in \Delta^m$ be source and target histograms, $\mathbf{M} \in \mathbb{R}_+^{n \times m}$ the ground cost (we use squared Euclidean distances), and the transport polytope be

$$\mathcal{U}(\mathbf{r}, \mathbf{c}) = \{\mathbf{P} \in \mathbb{R}_+^{n \times m} \mid \mathbf{P}\mathbf{1}_m = \mathbf{r}, \mathbf{P}^\top \mathbf{1}_n = \mathbf{c}\}. \tag{23}$$

The entropically regularized problem is

$$d_{M, \varepsilon}(\mathbf{r}, \mathbf{c}) = \min_{\mathbf{P} \in \mathcal{U}(\mathbf{r}, \mathbf{c})} \langle \mathbf{P}, \mathbf{M} \rangle - \varepsilon H(\mathbf{P}), \quad H(\mathbf{P}) = - \sum_{ij} P_{ij} (\log P_{ij} - 1), \tag{24}$$

whose solution has the scaling form $\mathbf{P}^* = \text{diag}(\mathbf{u}) \mathbf{K} \text{diag}(\mathbf{v})$ with the Gibbs kernel $\mathbf{K} = \exp(-\mathbf{M}/\varepsilon)$. The Sinkhorn iterations enforce

the marginal constraints:

$$\mathbf{u} \leftarrow \mathbf{r} \oslash (\mathbf{K}\mathbf{v}), \quad \mathbf{v} \leftarrow \mathbf{c} \oslash (\mathbf{K}^\top \mathbf{u}), \tag{25}$$

where $\oslash$ denotes element-wise division. This yields a differentiable map $(\mathbf{r}, \mathbf{c}, \mathbf{M}) \mapsto \mathbf{P}^*$. In our local OT setting, the source is a single point $\mathbf{r} = [1]$ and the target is a uniform distribution over n neighbors $\mathbf{c} = \frac{1}{n} \mathbf{1}$, so the optimal coupling degenerates to a softmax over costs:

$$\gamma_j = \frac{\frac{1}{n} e^{-M_{1j}/\varepsilon}}{\sum_{k=1}^n \frac{1}{n} e^{-M_{1k}/\varepsilon}} = \text{Softmax} \left(-\frac{1}{\varepsilon} [M_{11}, \dots, M_{1n}] \right)_j, \tag{26}$$

with the cost being $M_{1j} = \|\mathbf{x}_{\text{inj}} - \mathbf{x}_{\text{orig}}^j\|_2^2$. We compute $\mathcal{L}_{\text{OT}} = \langle \mathbf{P}^*, \mathbf{M} \rangle$ and backpropagate through a fixed number of Sinkhorn iterations, which are implemented in the log-domain for numerical stability.

B Training Algorithm

The complete training procedure of JANUS is outlined in Algorithm 1.

C Extended Experiments

This section provides additional experimental results to further underscore the robustness and superiority of the JANUS framework. We first analyze the attack transferability against victim models with heterogeneous architectures, then present a comprehensive visual comparison of feature-space stealthiness, and finally, evaluate the structural stealthiness by comparing graph-theoretic property distributions.

C.1 Performance on Heterogeneous Victim Architectures

A crucial aspect of a practical black-box attack is its ability to remain effective even when the victim model's architecture is unknown and differs from the surrogate model used for training. To test the transferability of JANUS, we evaluated its performance against a suite of victim models with diverse architectures: Graph Convolutional Network (GCN), Simple Graph Convolution (SGC), Graph Attention Network (GAT), and Approximated PageRank (APPNP).

The results, detailed in Table 5, strongly validate the black-box robustness of JANUS. Across all datasets and victim architectures, JANUS consistently and substantially outperforms the baselines. For instance, on the Pubmed dataset, it achieves a remarkable 91.1% misclassification rate against SGC and maintains an attack success rate of over 86% against all tested backbones—a level of performance unmatched by any competing method. Similarly, on Citeseer, while the effectiveness of baseline attacks fluctuates significantly depending on the victim model, JANUS consistently retains its state-of-the-art standing, achieving up to 67.5% success rate. This superior transferability stems from our dual-stealthiness constraints, which compel the model to learn the fundamental data distribution of the graph rather than overfitting to a specific message-passing scheme. This highlights the key advantage of our framework: JANUS generates attacks that are effective not because they exploit a particular model's weakness, but because they are intrinsically disruptive while remaining natural to the graph's underlying structure and features.

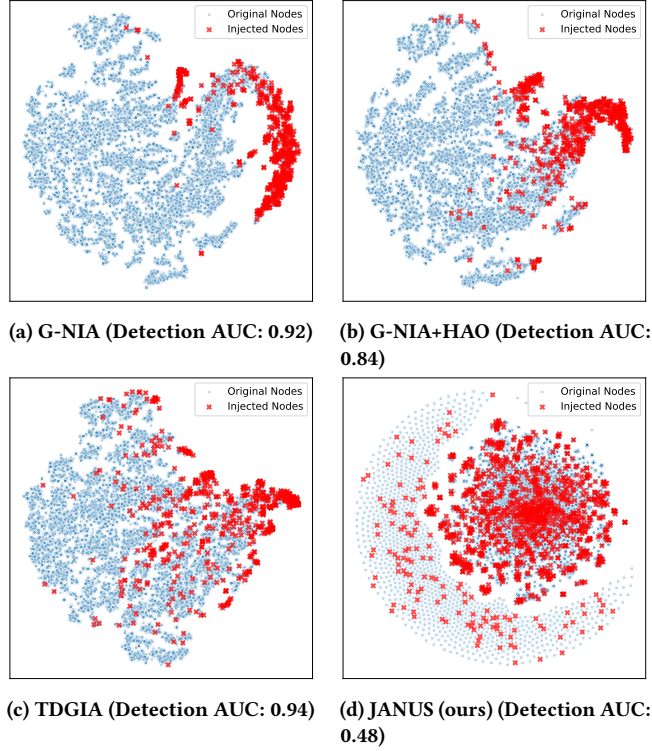


Figure 4: Extended t-SNE visualization and corresponding IForest detection AUC scores, comparing the feature space distribution of injected nodes (red) versus original nodes (blue) on the OGB-Products dataset.

Table 5: Misclassification rate (%) comparison against baselines on different two-layer GNN backbones.

Attacker	Backbone	Cora	Citeseer	Pubmed
G-NIA	GCN	24.5	40.5	68.1
	SGC	27.6	40.1	70.2
	GAT	22.1	35.3	68.8
	APPNP	24.2	37.2	69.7
TDGIA	GCN	31.2	44.2	70.9
	SGC	35.2	51.2	74.4
	GAT	28.5	52.1	81.0
	APPNP	29.1	44.0	70.1
G ² A2C	GCN	39.1	50.3	72.6
	SGC	41.6	53.8	76.3
	GAT	33.4	55.6	85.0
	APPNP	36.4	48.0	82.9
JANUS	GCN	60.7	66.9	89.5
	SGC	64.2	67.5	91.1
	GAT	46.7	56.3	86.9
	APPNP	47.3	53.5	87.2

C.2 Comparative Visualization of Injected Node Stealthiness

To provide a more intuitive and quantitatively-backed demonstration of JANUS’s stealthiness, we extend the t-SNE analysis. As the stealth-enhanced TDGIA+HAO baseline was discussed in the main paper, in Figure 4 we provide a direct visual comparison on the OGB-Products dataset between JANUS and three other representative baselines. We supplement each visualization with the Area Under the Curve (AUC) score from a trained Isolation Forest (IForest) detector, where an AUC of 0.5 represents random guessing. The analysis reveals a clear distinction in performance. The baseline methods, G-NIA [35], G-NIA+HAO, and TDGIA (Figures 4a-4c), all produce injected nodes that form visually distinct and dense clusters. This clear separation from the original node distribution (blue) is quantitatively reflected in their high IForest detection AUCs, confirming that their features are easily identifiable as anomalous. In stark contrast, JANUS (Figure 4d) demonstrates a fundamentally higher level of stealth. Visually, its injected nodes (red) are scattered and seamlessly integrated within the original data cloud, showing no discernible clustering or anomalous patterns. Quantitatively, this visual indistinguishability is powerfully corroborated by its near-random IForest AUC score of 0.48. This strong correspondence between the visual plot and the detection score proves that JANUS generates nodes that are authentic members of the feature manifold, not detectable outliers.

C.3 Analysis of Structural Property Distributions

Beyond feature-level stealth, a truly sophisticated attack must also ensure that its injected nodes are structurally indistinguishable from benign nodes. Naive injection strategies often create nodes with anomalous topological properties, making them easy to detect. To evaluate this crucial dimension of stealth, we select standard Closeness Centrality as our representative metric. This metric effectively quantifies how well-integrated an injected node is within the graph’s overall topology, making it a strong indicator for detecting structural anomalies.

Centrality metric. We use the standard closeness centrality as implemented by NetworkX on the post-injection, undirected, and unweighted graph. For a node v , let R_v be the set of nodes reachable from v . NetworkX returns the Wasserman-Faust improved normalization:

$$C_{NX}(v) = \frac{|R_v| - 1}{|V| - 1} \cdot \frac{|R_v| - 1}{\sum_{u \in R_v \setminus \{v\}} \text{dist}(v, u)} \quad (27)$$

which reduces to $(|V| - 1) / \sum_{u \neq v} \text{dist}(v, u)$ on connected graphs, and scales by the fraction of reachable nodes on disconnected graphs. Values lie in $[0, 1]$.

As seen in the Citeseer sub-figure (Figure 5a), the distribution of centrality for nodes injected by JANUS (orange curve) almost perfectly mimics that of the original nodes (blue curve). The two distributions exhibit a high degree of overlap, sharing nearly identical central tendencies, spread, and overall shape. This is in stark contrast to what would be expected from simpler attacks, which typically generate a large mass of nodes with near-zero centrality. We observe a similar, strong alignment on the large-scale OGB-Products dataset as well, confirming the robustness of this capability.

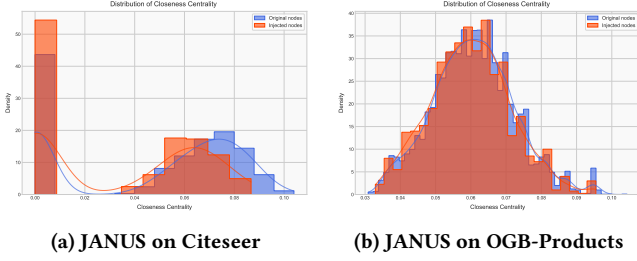


Figure 5: Distribution of Closeness Centrality for original nodes (blue) and nodes injected by JANUS (orange).

This exceptional structural mimicry is a direct consequence of JANUS’s global consistency constraint: by maximizing the $I(\mathbf{c}; \mathbf{g} | \mathbf{h})$ between structured latent codes and the generated structure, our framework forces the agent to move beyond myopic, local decisions. It learns the underlying structural patterns of the graph’s topology, ensuring that injected nodes are not just feature-wise plausible but are also seamlessly integrated into the structural fabric of the graph. This achieves a much deeper and more resilient level of stealth against sophisticated, structure-aware defense mechanisms.

C.4 Computational Cost Analysis

Theoretical Complexity Analysis. Let F_{GNN} denote the cost of a single forward pass of the victim GNN on the current sparse graph (for a 2-layer GCN, $F_{\text{GNN}} \approx O(|E|d)$). Per target, an episode injects at most Δ_n nodes and Δ_e edges per node. Each wiring step queries the victim once to compute the reward, yielding

$$O(\Delta_n \Delta_e \cdot F_{\text{GNN}}) \quad (28)$$

for the query component. The local OT loss uses an entropy-regularized Sinkhorn solver over a $1 \times m$ coupling (one injected feature vs. m sampled benign features). With L_{sink} iterations, its cost is $O(\Delta_n \cdot$

$L_{\text{sink}} \cdot m)$ and is independent of $|E|$. Hence, the total per-target complexity is

$$O(\Delta_n \Delta_e \cdot F_{\text{GNN}} + \Delta_n L_{\text{sink}} m) \quad (29)$$

where the first term dominates on large graphs. The discriminator and the Q-head run on mini-batch graph embeddings and add only sub-dominant overhead independent of $|E|$; hence the overall order is on par with other RL-based black-box methods (e.g., G^2A2C), while our OT and node generator modules contribute only lightweight costs.

Empirical Runtime Comparison. Protocol and analysis. To ensure a fair comparison of computational overhead against G^2A2C , which is also a reinforcement learning-based framework, we established a controlled protocol. A direct comparison of total training time can be misleading, as the total amount of optimization before early stopping is triggered may vary between methods. To eliminate this variability from inconsistent optimization steps, we ensured that both methods attack the same number of nodes per epoch and employ identical early stopping criteria. Consequently, we report the average wall-clock time per epoch, a fairer metric that better reflects the inherent complexity of the algorithm per iteration. As shown in Table 6, the empirical results on the Citeseer dataset show that JANUS adds only ≈ 1.56 s/epoch over G^2A2C while improving the misclassification rate by +16.6 points; the absolute per-epoch time is modest for black-box graph attacks.

Table 6: Illustrative computational cost comparison on Citeseer.

Attacker	Attack Time (s/epoch) ↓	Misclass. Rate (%) ↑
G^2A2C	6.51	50.3
JANUS	8.07	66.9